E1

Wednesday, February 18, 2026 3:27 PM

ERP rendszer - rabarburaERP
Odoo - framework
Odoo version 7-8 - (openRP regi neven)-> modified
postgreSQL - 13.5
Linux servers
Ubuntu 24.04.3 (currently)
Fejlesztes linuxon
Minden sajat
ERP rendszerek: - specko vagy integralt modulok
Gyartasvezerles, hr management, dokumentum kezeles
Uj trend: automatizalt konyveles

2. N8n - workflow, automatizalasi rendszer/platform integralva sajat modulokkal
   (make.com - legnagyobb automatizalasi rendszer)

3. Eoflex - sajat product
   Fully ai assisted
   Python, fastAPI
   Backend:SQL Alchemy orm, Alembic migrations, Pydantic validation
   Frontend: ninjaJS

   Claude code - ai amit hasznalnak - max elofizetes
   Abacus.ai - conversation

4. euFlex DocSign

Vagy odoo cucc vagy ai assisted cucc

Openrouter.ai - - > antropic
Llama 4 maverick - used

Docker, adatbazis, user interface, ai-al

Windows powershell run app:

python -m venv .venv
.\\.venv\\Scripts\\Activate.ps1
pip install -r requirements.txt
uvicorn backend.app.main:app --reload

In another terminal (to serve frontend):
cd frontend
npx serve -s . -l 5173

Running smoke test locally:

1. Create venv and install deps:

   python -m venv .venv
   .\\.venv\\Scripts\\Activate.ps1

pip install -r requirements.txt

2. Start the API(terminal A):

   uvicorn backend.app.main:app --reload

3. Running the smoke test:
   python tests/smoke_test.py

Best Practice for Modern Apps

Recommended stack:

```
CarFleetManagement
 ├── backend
 │   └── ASP.NET Core Web API
 │
 └── frontend
     └── React / Angular SPA
```

Backend = **data + business logic**

Frontend = **UI + API calls**

Prompting

Monday, March 9, 2026 4:37 PM

Create all the necessary files, classes, methods, database connections etc. for the Car Fleet Management application.
The necessary classes for this project contain:

- Vehicle.cs (this file already exists)
- Drivers or Employees class, which should include the following attributes:
  - Id (PK)
  - FirstName
  - LastName
  - LicenseNumber
  - LicenseExpiryDate
  - Phone
  - Email
  - HireDate
  - Status
- VehicleAssignment class, tracks which driver uses which vehicle
  - Id (PK)
  - VehicleId (FK)
  - DriverId (FK)
  - StartDate
  - EndDate
  - AssignedBy
  - Notes

- Fuelogs class for tracking fuel logs ang consumption
  - Id (PK)
  - VehicleId (FK)
  - DriverId (FK)
  - FuelDate
  - Liters
  - PricePerLiter
  - TotalCost
  - Odometer
  - FuelStation

- ServiceRecords for tracking maintenance and repairs
  - Id (PK)
  - VehicleId (FK)
  - ServiceType
  - Description
  - ServiceDate
  - Odometer
  - Cost
  - ServiceProvider
  - NextServiceDue

- Expenses class for keeping track of other expenses not related to fuel
  - Id (PK)
  - VehicleId (FK)

ExpenseType
Amount
ExpenseDate
Vendor
Description

- Trips class for tracking journeys
  Id (PK)
  VehicleId (FK)
  DriverId (FK)
  StartTime
  EndTime
  StartLocation
  EndLocation
  DistanceKm
  Purpose

- Documents class for storing vehicle documents
  Id (PK)
  VehicleId (FK)
  DocumentType
  FilePath
  IssueDate
  ExpiryDate
  UploadedAt

The following is the relationship diagram between the classes:

```
Drivers
  |
  |
VehicleAssignments
  |
  |
Vehicles
 |   |   |   |
 |   |   |   └──── Documents
 |   |   |
 |   |   └──────── ServiceRecords
 |   |
 |   └──────────── FuelLogs
 |
 └──────────────── Trips
```

The application should have the following features:

Feature	**Table**
vehicle inventory	Vehicles
driver management	Drivers
vehicle usage	Assignments

fuel costs	FuelLogs
maintenance	ServiceRecords
operational costs	Expenses
trips	Trips

The key functionalities of the application are the following:

You are tasked with designing and implementing a **Car Fleet Management System backend** written in **C# using ASP.NET Core, Entity Framework Core, and PostgreSQL**.
The system should follow **clean architecture principles** and expose a **REST API**.
Your goal is to generate the architecture, database models, API endpoints, and services needed to implement the following functionality.

SYSTEM OVERVIEW

The application manages a company's vehicle fleet.
It must track vehicles, drivers, assignments, fuel usage, maintenance, trips, expenses, and documents.
The backend must support CRUD operations, reporting, authentication, and notifications.
Use:

- ASP.NET Core Web API
- Entity Framework Core
- PostgreSQL
- Dependency Injection
- Repository / Service pattern
- DTOs
- Swagger API documentation

CORE ENTITIES

Implement database models and relationships for the following entities:
Vehicles
Drivers
VehicleAssignments
FuelLogs
ServiceRecords
Trips
Expenses
Documents
FleetLocations
Users
Roles
Relationships should include:

- Vehicles ↔ Drivers (many-to-many via VehicleAssignments)
- Vehicles ↔ FuelLogs
- Vehicles ↔ ServiceRecords
- Vehicles ↔ Trips
- Vehicles ↔ Expenses
- Vehicles ↔ Documents
- Drivers ↔ Trips

EPIC 1 — VEHICLE MANAGEMENT

Features:

- Register new vehicle
- Update vehicle details
- View vehicle list
- View vehicle details
- Delete vehicle
- Mark vehicle as retired or inactive
- Search vehicles
- Filter vehicles by make, status, or year

Tasks:

- Implement Vehicle entity
- Create Vehicle DTOs
- Implement repository layer
- Implement service layer
- Implement VehicleController
- Add validation
- Add pagination

EPIC 2 — DRIVER MANAGEMENT

Features:

- Add driver
- Edit driver information
- View driver profile
- Remove driver
- Track driver license expiration
- Search drivers

Tasks:

- Implement Driver entity
- Create Driver CRUD API
- Implement driver validation
- License expiration tracking

EPIC 3 — VEHICLE ASSIGNMENTS

Features:

- Assign vehicle to driver
- Remove assignment
- View assignment history
- Prevent multiple active assignments

Tasks:

- Implement VehicleAssignment entity
- Create assignment endpoints
- Validate assignment rules

EPIC 4 — FUEL MANAGEMENT

Features:

- Log fuel purchases
- Track fuel cost
- Track fuel consumption
- Calculate fuel efficiency

Tasks:

- Implement FuelLog entity
- Fuel log CRUD API

- Fuel consumption calculations

EPIC 5 — MAINTENANCE MANAGEMENT

Features:

- Record maintenance events
- Track repair history
- Track service providers
- Schedule maintenance
- Track maintenance costs

Tasks:

- Implement ServiceRecord entity
- Maintenance CRUD API
- Service reminders

EPIC 6 — TRIP TRACKING

Features:

- Log trips
- Track trip distance
- Record start and end locations
- Assign drivers to trips
- Calculate trip cost

Tasks:

- Implement Trip entity
- Trip CRUD API

EPIC 7 — EXPENSE MANAGEMENT

Features:

- Log vehicle expenses
- Categorize expenses
- Generate expense reports

Tasks:

- Implement Expense entity
- Expense CRUD API

EPIC 8 — DOCUMENT MANAGEMENT

Features:

- Upload vehicle documents
- Store document metadata
- Track expiration dates
- Download documents

Examples:
Insurance
Registration
Inspection certificates
Tasks:

- Implement Document entity
- File upload API
- Document expiration alerts

EPIC 9 — REPORTING AND ANALYTICS

Features:

- Fleet dashboard
- Fuel consumption reports
- Maintenance cost reports
- Vehicle utilization metrics

Tasks:

- Reporting endpoints
- Aggregate queries

EPIC 10 — AUTHENTICATION AND AUTHORIZATION

Features:

- User registration
- Login
- JWT authentication
- Role-based authorization

Roles:
Admin
FleetManager
Driver
Tasks:

- Implement User entity
- Implement Role entity
- Implement JWT authentication

EPIC 11 — NOTIFICATIONS

Features:

- Maintenance reminders
- License expiration alerts
- Insurance expiration alerts

Tasks:

- Background job scheduler
- Email notifications

EPIC 12 — API INFRASTRUCTURE

Features:

- REST API
- API versioning
- Global exception handling
- Request validation
- Logging

EPIC 13 — TESTING

Implement:

- Unit tests
- Integration tests
- API tests

Use xUnit for testing.

OUTPUT REQUIREMENTS

When implementing the system, generate:

1. Database models
2. Entity Framework DbContext
3. Migrations
4. Repository layer
5. Service layer
6. Controllers
7. DTOs
8. Validation
9. Unit tests
10. API documentation

Ensure the architecture is modular, maintainable, and scalable.